

Hallucination Detection

& Fact Verification API

A full-stack app that checks whether an LLM's answer is actually supported by its source context — decomposes the answer into atomic claims, judges each one against the source, and returns a risk score, verdict, and quoted evidence for every claim.

The Stack

Two separate services: a Next.js frontend and a FastAPI backend, talking over plain REST.

FRONTEND

Next.js 16

App Router + Turbopack, React 19, TypeScript. Six client-rendered pages: Dashboard, Studio, Analytics, Developer API, Settings.

Tailwind CSS v4

Utility-first styling; class-based dark mode via a custom variant, toggled and persisted client-side.

Recharts

Line/bar/pie charts on Dashboard & Analytics — hallucination rate over time, error-type breakdown, engine mix.

Axios

REST client with an interceptor that attaches the Bearer API key and auto-retries once on a stale-key 401.

BACKEND

FastAPI

Python REST API — routers for verify, settings, api-keys, analytics. Auto-generates OpenAPI docs at `/docs`.

Uvicorn

ASGI server running the FastAPI app on port 8000.

Pydantic

Typed request/response schemas (`Claim` , `Verdict` , `ErrorType` enums) — validates every payload automatically.

python-dotenv

Loads `GEMINI_API_KEY` , model name, and thresholds from a local `.env` file.

AI LAYER

DATA & AUTH

Google Gemini API

Model `gemini-3.5-flash-lite`, via the `google-generativeai` SDK. One JSON-mode call per verification.

Rule-based fallback

Hand-written regex/heuristic classifier — zero external calls, always available, used when Gemini fails or is rate-limited.

JSON file storage

No database — `history.json`, `settings.json`, `api_keys.json` read/written directly on disk.

API-key auth

Custom Bearer-token scheme; a key auto-issues on first page load so the demo needs zero setup.

How Verification Works

What actually happens between clicking "Run" and seeing a verdict.

1 Studio sends context + answer

The frontend POSTs `{ context, answer, model }` to `/v1/verify`, authenticated with the Bearer key.

2 One combined Gemini call: decompose + classify

A single prompt asks Gemini to (a) split the answer into atomic, independently-checkable claims and (b) judge each one against the context — returning a risk score (0–100), an error type, and quoted evidence, all in one JSON response.

3 Risk score → verdict, via configurable thresholds

Each claim's score is compared against the Settings page's lower/upper bounds (default 40 / 60): below lower → **Supported**, above upper → **Contradicted**, in between → **Abstain**.

4 Fallback on any Gemini failure

If the API call fails for any reason (quota, network, malformed response), the whole verification — not just individual claims — falls back to the deterministic rule-based classifier below, so results are never a mix of both methods.

5 Logged for Analytics

Every run is appended to `history.json` with its model tag and engine used (`gemini` / `rule_based_fallback`), which the Analytics/Dashboard pages aggregate live.

Fallback Classifier — What It Catches Without Any API Call

A safety net, not a guess — five targeted checks run in order:

numerical_mismatch A number in the claim doesn't appear anywhere in the context — *or* the number matches but its qualifier doesn't (currency: USD vs. INR; magnitude: million vs. billion; unit: km vs. miles).

year_in_context A year in the claim isn't in the context

- temporal_mismatch** A year in the claim isn't in the context.
- entity_mismatch** A proper noun in the claim (person, place, org) doesn't appear anywhere in the context.
- contradiction** The claim and its best-matching context sentence disagree on negation/polarity — e.g. claim says "will guarantee," context says "not guaranteed."
- unverifiable** Word-overlap with the context is too low to confirm or deny the claim at all.

Request Flow



Next.js has **no server-side API routes of its own** — it's a pure client. Every bit of verification logic lives in the separate FastAPI service, reached over CORS-enabled REST at `localhost:8000`.

API Surface

METHOD	PATH	PURPOSE
POST	/v1/verify	Decompose + classify an answer against a context
GET/PUT	/v1/settings	Read/update risk-threshold calibration
GET/POST	/v1/api-keys	List / create keys (+ roll, revoke)
GET	/v1/analytics/*	summary , timeseries , models
GET	/v1/health	Liveness check

POST /V1/VERIFY – REQUEST

```
{
  "context": "Q3 revenue was $45M.",
```

– RESPONSE

```
{
  "claims": [{
```

```

    "answer": "Revenue hit $50M in Q3.",
    "model": "gpt-4o-mini"
  },
  {
    "text": "Revenue hit $50M in Q3.",
    "verdict": "Contradicted",
    "risk_score": 88.0,
    "error_type": "numerical_mismatch"
  }
],
"hallucination_rate": 100.0,
"engine": "gemini"
}

```

Data Files

history.json Every verification run: timestamp, model, source, engine used, and each claim's verdict — feeds the Analytics/Dashboard charts directly.

settings.json The two risk-threshold values (default 40/60) that convert a raw risk score into Supported/Abstain/Contradicted.

api_keys.json Issued Bearer keys — label, preview, created/last-used timestamps, revoked flag.

Design Decisions (Q&A Prep)

Why FastAPI + Next.js instead of Next.js API routes?

Keeps the AI/verification logic in Python (better LLM/SDK ecosystem) fully decoupled from the UI — either side can be redeployed, tested, or replaced independently.

Why JSON files instead of a real database?

Right-sized for a single-demo project — no schema migrations, easy to inspect/reset by hand. The API surface (`history_service` , `settings_service`) is already isolated, so swapping in SQLite/Postgres later wouldn't touch the routers.

Why one combined Gemini call instead of one call per claim?

Cuts API usage roughly 5x (critical on a rate-limited free tier) and guarantees every claim in an answer is judged by the same method — never half-Gemini, half-fallback within one response.

Why keep a non-AI fallback at all?

Resilience: a live demo shouldn't go dark because of a quota limit or network blip. It's deliberately conservative — it flags four concrete error types and never fabricates confidence it doesn't have.

Before You Present

1. **Enable billing** on the Gemini key beforehand — the free tier is thin and you don't want to hit it live.
2. **Have two pre-tested examples ready** — one clearly Supported, one clearly Contradicted — as a backup.
3. **Settings page is a real lever**, not decoration — the sliders directly change verdict boundaries live.